CS302: SCITAS Guide

Introduction

This guide provides the essential steps to access SCITAS, which will be needed for this class assignments. It covers server connection, environment setup, and how to run jobs.

Connect to the server

List of Servers Available

You will use different clusters depending on the assignment. Ensure to use the correct one.

Assignment 1 and 2	`jed.hpc.epfl.ch`
Assignment 3	`izar.hpc.epfl.ch`

Key Setup

To connect securely without a password, you must set up an SSH key. As you likely already have at least one key (e.g., for GitHub), we show how to install multiple keys on the same machine.

1. Create the directory structure

```
mkdir -p ~/.ssh/keys
```

2. Generate a new key per server (e.g., `jed` and `izar`)

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/keys/jed
```

This command creates a private key (`jed`) and a public key (`jed.pub`) in your `~/.ssh/keys` directory.

3. Add it to the config

Edit your SSH configuration file (~/.ssh/config) to add your keys (like GitHub and `jed`)

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/keys/github
  IdentitiesOnly yes

Host jed
  HostName jed.hpc.epfl.ch
  User <username>
  Port 22
  IdentityFile ~/.ssh/keys/jed
  IdentitiesOnly yes
```

Replace `<username>` with your EPFL username (typically the first letter of your first name concatenated with your last name) and `HostName` with the correct server (see above).

4. Upload the key on the server

```
ssh-copy-id -i ~/.ssh/keys/jed.pub <username>@jed.hpc.epfl.ch
```

You will be prompted for your EPFL password. Once completed, you can log in without it.

Integration with VS-Code (optional)

If you are using VS-Code, you can use the `Remote - SSH` extension to directly edit code on the remote server.

1. Install the extension

Go to the Extension view in VS-Code (on the left panel) and install the extension
`Remote - SSH`

2. Connect
   - On the left panel, click on the `Remote Explorer` tab.
   - Select `jed` (or any remote server listed in your config) from the list.

Job Scheduling

SCITAS uses the *Slurm* workload manager. The idea is to submit jobs that the scheduler executes on compute nodes.

File Directories

Use `/scratch/<username>` since it's optimized for high performance operations. Its content may be removed every 30 days in case you are using a large amount of storage, so make sure to back up important files and clean up large output files regularly.

Writing a Job Script

Go to /scratch/<username> and create a new file named `execute.sh`. Below is an example template.

```
#!/usr/bin/env bash
#SBATCH --account cs-302
#SBATCH --qos cs-302
#SBATCH --partition academic
#SBATCH --nodes 1
#SBATCH --ntasks-per-node 1
#SBATCH --cpus-per-task 32
#SBATCH --mem 32G
#SBATCH --time 00:30:00

module purge
module load gcc
module load openmpi

echo STARTING AT `date`

# compile
# ... make ...

# execute
srun <prog> <args>

echo FINISHED at `date`
```

Notes:

- **Always compile the code on the server** to avoid any library version mismatch.

SBATCH Options

The job template contains the most important parameters for the assignments.

`#SBATCH --qos cs-302`	Use `cs-302` for up to 2 nodes (with high priority) - time limit: 3 hours
`#SBATCH --nodes 1`	Number of compute nodes (i.e., cpus) requested.
`#SBATCH --ntasks-per-node 1`	Number of tasks (i.e., processes) to start per node.
`#SBATCH --cpus-per-task 32`	Number of thread per process

`#SBATCH --mem 32G`	Amount of RAM requested (per node) - Be reasonable. Asking for too much increases queue wait time.
`#SBATCH --time 00:30:00`	Time limit (HH:MM:SS) - Be reasonable. Shorter times decrease queue wait time.

Notes:

- For Assignment 1, `ntasks-per-node = 1`
- For Assignment 2, `cpus-per-task = 1`, except for the last part where you combine OMP with MPI. In that case, you need to tweak the parameters to achieve best results. Keep in mind the following constraint (per node):
  - `ntasks-per-node x cpus-per-task <= 72`

Submit a Job

To submit your job to the queue, run:

```
sbatch execute.sh
```

DO NOT simply execute the script (i.e., with `./execute.sh`), since you won't use the compute nodes and the hardware available may be limited.

Monitor a job

To check the status of your jobs, use:

```
squeue -u $USER
```

When the job is done, it disappears from the list.

View Results

When the job starts running, a new file with the name `slurm-<JOBID>.out` is created in the directory where you submitted the job. You can view it with:

```
cat slurm-<JOBID>.out
```

To track live progress through your logs, use:

```
tail -f slurm-<JOBID>.out
```

Cancel a job

If you realized that you made a mistake and want to submit a new job, first make sure that all of your running jobs are stopped. You don't want to waste compute resources.

To do so, first identify the job id, using the `squeue` command from the Monitor a Job section. Then, execute:

```
scancel <id>
```